

```
In [1]: from dsc80_utils import *
```

Lecture 18 – Classifier Evaluation, Model Fairness

DSC 80, Fall 2025

Agenda

- Classifier evaluation.
- Logistic regression.
- Model fairness.

Aside: [MLU Explain](#) is a **great** resource with visual explanations of many of our recent topics (cross-validation, random forests, precision and recall, etc.).

Random Forests

```
In [2]: diabetes = pd.read_csv(Path('data') / 'diabetes.csv')
display_df(diabetes, cols=9)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigree
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
...	...	...	...	...	...	...	
765	5	121	72	23	112	26.2	
766	1	126	60	0	0	30.1	
767	1	93	70	31	0	30.4	

768 rows × 9 columns

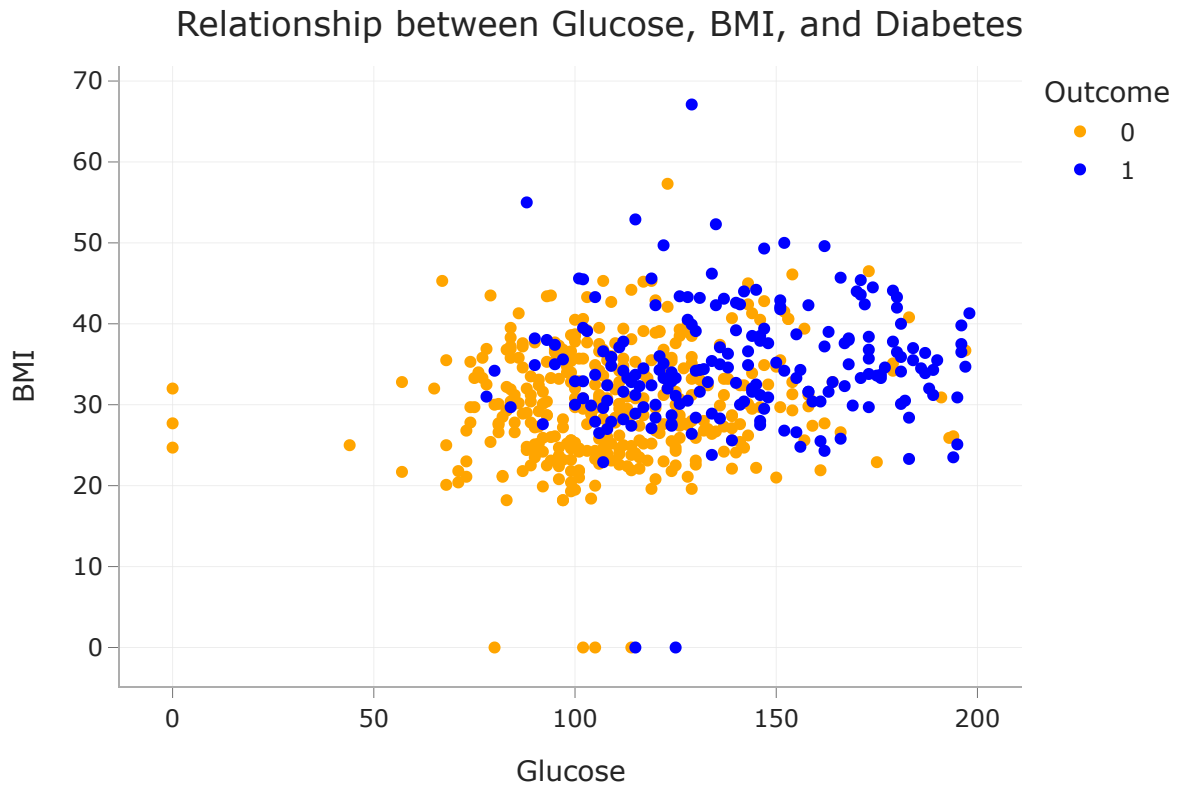
```
In [3]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = (
    train_test_split(diabetes[['Glucose', 'BMI']], diabetes['Outcome'], rand
)
```

```
In [4]: fig = (
    plt.figure(figsize=(10, 5))
    plt.title('Diabetes Outcome by BMI')
    plt.xlabel('BMI')
    plt.ylabel('Outcome')
    plt.scatter(X_train['BMI'], y_train, s=100)
    plt.grid(True)
    plt.show()
)
```

```

.plot(kind='scatter', x='Glucose', y='BMI', color='Outcome',
      color_discrete_map={'0': 'orange', '1': 'blue'},
      title='Relationship between Glucose, BMI, and Diabetes')
)
fig

```



Random Forests

Main idea:

Train a bunch of decision trees, then have them **vote** on a prediction!

- **Problem:** If you use the same training data, you will always get the same tree.
- **Solution:** Introduce randomness into training procedure to get different trees.

Idea 1: Bootstrap the training data

- We can bootstrap the training data B times, then train one tree on each resample.
- Also known as bagging (Bootstrap AGgregating). In general, combining different predictors together is a useful technique called **ensemble learning**.
- For decision trees though, doesn't make trees different enough from each other (e.g. if you have one really strong predictor, it'll always be the first split).

Idea 2: Only use a subset of features

Loading [MathJax]/extensions/Safe.js

- At each split, take a random subset of m features instead of choosing from all d of them.
- Rule of thumb: $m \approx \sqrt{d}$ seems to work well.
- Key idea: For ensemble learning, you want the individual predictors to have low bias, high variance, and be uncorrelated with each other. That way, when you average them together, you have low bias AND low variance.
- **Random forest algorithm:** Fit T trees by using bagging and a random subset of features at each split. Predict by taking a vote from the T trees.

Question 🤔

How will increasing m affect the complexity of the model?

Example

```
In [5]: # Let's use more features for prediction
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = (
    train_test_split(diabetes.drop(columns=['Outcome']), diabetes['Outcome'])
)
```

```
In [6]: from sklearn.ensemble import RandomForestClassifier

clf = RandomForestClassifier()
clf.fit(X_train, y_train)
clf.score(X_train, y_train)
```

Out[6]: 1.0

```
In [7]: clf.score(X_test, y_test)
```

Out[7]: 0.828125

Compared to our previous best decision tree with depth 4:

```
In [8]: from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(max_depth=4, criterion='entropy')
dt.fit(X_train, y_train)
dt.score(X_train, y_train)
```

Out[8]: 0.7829861111111112

```
In [9]: dt.score(X_test, y_test)
```

Loading [MathJax]/extensions/Safe.js

Out [9]: 0.734375

Example: Modeling using text features

Example: Fake news

We have a dataset containing news articles and labels for whether the article was deemed "fake" or "real". Credit to <https://github.com/KaiDMML/FakeNewsNet>.

```
In [10]: news = pd.read_csv('data/fake_news_training.csv')
news
```

```
Out [10]:
```

	baseurl	content	label
0	twitter.com	\njavascript is not available.\n\nwe've detect...	real
1	whitehouse.gov	remarks by the president at campaign event -- ...	real
2	web.archive.org	the committee on energy and commerce\nbarton: ...	real
...	...	...	...
658	politico.com	full text: jeff flake on trump speech transcri...	fake
659	pol.moveon.org	moveon.org political action: 10 things to know...	real
660	uspostman.com	uspostman.com is for sale\nyes, you can transf...	fake

661 rows × 3 columns

Goal: Use an article's content to predict its label.

```
In [11]: news['label'].value_counts(normalize=True)
```

```
Out [11]: label
real      0.55
fake      0.45
Name: proportion, dtype: float64
```

Question: What is the baseline accuracy you'd expect here? That is, imagine a model that always predicts "real". What is its accuracy?

Aside: CountVectorizer

Entries in the 'content' column are not currently quantitative! We can use the bag of words encoding to create quantitative features out of each 'content'.

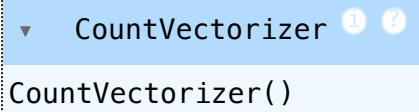
Instead of performing a bag of words encoding manually as we did before, we can rely on `sklearn`'s `CountVectorizer`. (There is also a `TfidfVectorizer`.)

Loading [MathJax]/extensions/Safe.js

```
In [12]: from sklearn.feature_extraction.text import CountVectorizer
```

```
In [13]: example_corp = ['hey hey hey my name is billy',
                        'hey billy how is your dog billy']
```

```
In [14]: count_vec = CountVectorizer()
count_vec.fit(example_corp)
```

```
Out[14]: 
CountVectorizer()
```

`count_vec` learned a **vocabulary** from the corpus we `fit` it on.

```
In [15]: count_vec.vocabulary_
```

```
Out[15]: {'hey': 2,
          'my': 5,
          'name': 6,
          'is': 4,
          'billy': 0,
          'how': 3,
          'your': 7,
          'dog': 1}
```

```
In [16]: count_vec.transform(example_corp).toarray()
```

```
Out[16]: array([[1, 0, 3, 0, 1, 1, 1, 0],
                [2, 1, 1, 1, 1, 0, 0, 1]])
```

Note that the values in `count_vec.vocabulary_` correspond to the positions of the columns in `count_vec.transform(example_corp).toarray()`, i.e. `'billy'` is the first column and `'your'` is the last column.

```
In [17]: example_corp
```

```
Out[17]: ['hey hey hey my name is billy', 'hey billy how is your dog billy']
```

```
In [18]: pd.DataFrame(count_vec.transform(example_corp).toarray(),
                      columns=pd.Series(count_vec.vocabulary_).sort_values().index)
```

```
Out[18]:
```

	billy	dog	hey	how	is	my	name	your
0	1	0	3	0	1	1	1	0
1	2	1	1	1	1	0	0	1

Creating an initial Pipeline

Loading [MathJax]/extensions/Safe.js Pipeline that takes in summaries and overall ratings and:

- Uses `CountVectorizer` to quantitatively encode summaries.
- Fits a `RandomForestClassifier` to the data.

But first, a train-test split (like **always**).

```
In [19]: from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
```

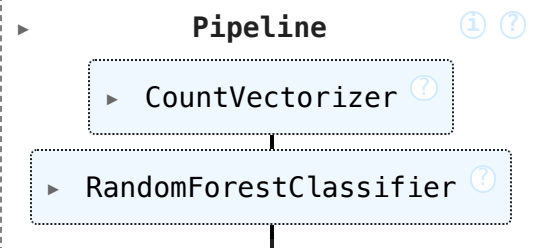
```
In [20]: X = news['content']
y = news['label']
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

To start, we'll create a random forest with 100 trees (`n_estimators`) each of which has a maximum depth of 3 (`max_depth`).

```
In [21]: pl = Pipeline([
    ('cv', CountVectorizer()),
    ('clf', RandomForestClassifier(
        max_depth=3,
        n_estimators=100, # Uses 100 separate decision trees!
        random_state=42,
    ))
])
```

```
In [22]: pl.fit(X_train, y_train)
```

```
Out [22]:
```



```
In [23]: # Training accuracy.
pl.score(X_train, y_train)
```

```
Out [23]: 0.7555555555555555
```

```
In [24]: # Testing accuracy.
pl.score(X_test, y_test)
```

```
Out [24]: 0.6506024096385542
```

The accuracy of our random forest is just under 73%, on the test set. How much better does it do compared to a classifier that predicts "real" every time?

```
In [25]: y_train.value_counts(normalize=True)
```

Loading [MathJax]/extensions/Safe.js

```
Out[25]: label
        real    0.52
        fake    0.48
        Name: proportion, dtype: float64
```

```
In [26]: # Distribution of predicted ys in the training set:

# stops scientific notation for pandas
pd.set_option('display.float_format', '{:.3f}'.format)
pd.Series(pl.predict(X_train)).value_counts(normalize=True)
```

```
Out[26]: fake    0.701
        real    0.299
        Name: proportion, dtype: float64
```

```
In [27]: len(pl.named_steps['cv'].vocabulary_) # Lots of features!
```

```
Out[27]: 27129
```

Choosing tree depth via GridSearchCV

We arbitrarily chose `max_depth=3` before, but it seems like that isn't working well. Let's perform a grid search to find the `max_depth` with the best generalization performance.

```
In [28]: # Note that we've used the key clf__max_depth, not max_depth
# because max_depth is a hyperparameter of clf, not of pl.

hyperparameters = {
    'clf__max_depth': np.arange(2, 200, 20)
}
```

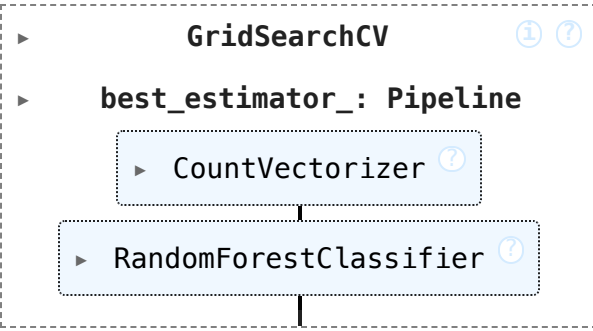
Note that while `pl` has already been `fit`, we can still give it to `GridSearchCV`, which will repeatedly re-`fit` it during cross-validation.

```
In [29]: %%time

# Takes a few seconds to run – how many trees are being trained?
from sklearn.model_selection import GridSearchCV
grids = GridSearchCV(
    pl,
    n_jobs=-1, # Use multiple processors to parallelize
    param_grid=hyperparameters,
    return_train_score=True
)
grids.fit(X_train, y_train)
```

```
CPU times: user 1.1 s, sys: 158 ms, total: 1.26 s
Wall time: 6.66 s
```

```
Out [29]:
```



```
In [30]: grids.best_params_
```

```
Out [30]: {'clf__max_depth': np.int64(82)}
```

Recall, `fit` `GridSearchCV` objects are estimators on their own as well. This means we can compute the training and testing accuracies of the "best" random forest directly:

```
In [31]: # Training accuracy.
grids.score(X_train, y_train)
```

```
Out [31]: 0.9959595959595959
```

```
In [32]: # Testing accuracy.
grids.score(X_test, y_test)
```

```
Out [32]: 0.7710843373493976
```

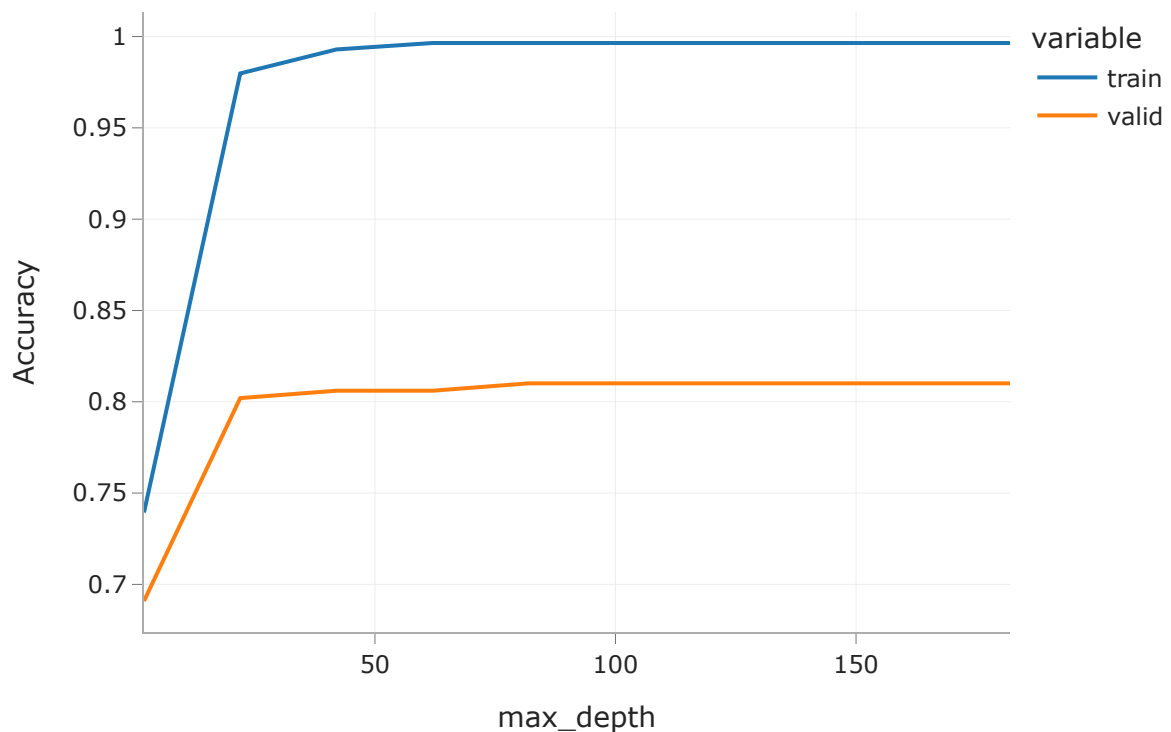
~10% better test set error!

Training and validation accuracy vs. depth

Below, we plot how training and validation accuracy varied with tree depth. Note that the y -axis here is accuracy, and that larger accuracies are better (unlike with RMSE, where smaller was better).

```
In [33]: index = grids.param_grid['clf__max_depth']
train = grids.cv_results_['mean_train_score']
valid = grids.cv_results_['mean_test_score']
```

```
In [34]: pd.DataFrame({'train': train, 'valid': valid}, index=index).plot().update_layout(
    xaxis_title='max_depth', yaxis_title='Accuracy'
)
```

Question 🤔

(Fa23 Final Q10.2)

Suppose we write the following code:

```
hyperparameters = {
    'n_estimators': [10, 100, 1000], # number of trees per forest
    'max_depth': [None, 100, 10]     # max depth of each tree
}
grids = GridSearchCV(
    RandomForestClassifier(), param_grid=hyperparameters,
    cv=3, # 3-fold cross-validation
)
grids.fit(X_train, y_train)
```

Answer the following questions with a single number.

1. How many random forests are fit in total?
2. How many decision trees are fit in total?
3. How many times in total is the first point in X_{train} used to train a decision tree?

Classifier Evaluation (this section is on Exam 02!)

$$\text{accuracy} = \frac{\text{\# data points classified correctly}}{\text{\# data points}}$$

- Accuracy is defined as the proportion of predictions that are correct.
- It weighs all **correct** predictions the same, and weighs all **incorrect** predictions the same.
- But some incorrect predictions may be worse than others!
 - Example: Suppose you take a COVID test 🦠. Which is worse:
 - **False positive:** The test **saying you have COVID**, when you really don't, or
 - **False negative:** The test **saying you don't have COVID**, when you really do?

The Boy Who Cried Wolf 🧑🏻🥲🐺

([source](#))

A shepherd boy gets bored tending the town's flock. To have some fun, he cries out, "Wolf!" even though no wolf is in sight. The villagers run to protect the flock, but then get really mad when they realize the boy was playing a joke on them.

Repeat the previous paragraph many, many times.

One night, the shepherd boy sees a real wolf approaching the flock and calls out, "Wolf!" The villagers refuse to be fooled again and stay in their houses. The hungry wolf turns the flock into lamb chops. The town goes hungry. Panic ensues.

The wolf classifier

- Predictor: Shepherd boy.
- Positive prediction: "There is a wolf."
- Negative prediction: "There is no wolf."

Some questions to think about:

- What is an example of an incorrect, positive prediction?
- Was there a correct, negative prediction?

- There are four possibilities. What are the consequences of each?
 - (predict yes, predict no) x (actually yes, actually no).

The wolf classifier

Below, we present a **confusion matrix**, which summarizes the four possible outcomes of the wolf classifier.

True Positive (TP): • Reality: A wolf threatened. • Shepherd said: "Wolf." • Outcome: Shepherd is a hero.	False Positive (FP): • Reality: No wolf threatened. • Shepherd said: "Wolf." • Outcome: Villagers are angry at shepherd for waking them up.
False Negative (FN): • Reality: A wolf threatened. • Shepherd said: "No wolf." • Outcome: The wolf ate all the sheep.	True Negative (TN): • Reality: No wolf threatened. • Shepherd said: "No wolf." • Outcome: Everyone is fine.

Outcomes in binary classification

When performing **binary** classification, there are four possible outcomes.

(Note: A "positive prediction" is a prediction of 1, and a "negative prediction" is a prediction of 0.)

Outcome of Prediction	Definition	True Class
True positive (TP) ✓	The predictor correctly predicts the positive class.	P
False negative (FN) ✗	The predictor incorrectly predicts the negative class.	P
True negative (TN) ✓	The predictor correctly predicts the negative class.	N
False positive (FP) ✗	The predictor incorrectly predicts the positive class.	N



	Predicted Negative	Predicted Positive
Actually Negative	TN ✓	FP ✗
Actually Positive	FN ✗	TP ✓

The **confusion matrix** above is organized the same way that `sklearn`'s confusion matrices are (but differently than in the wolf example).

Note that in the four acronyms – TP, FN, TN, FP – the **first letter** is whether the prediction is correct, and the **second letter** is what the prediction is.

Example: COVID testing

- UCSD Health administers hundreds of COVID tests a day. The tests are not fully accurate.
- Each test comes back either
 - positive, indicating that the individual has COVID, or
 - negative, indicating that the individual does not have COVID.
- Question:** What is a TP in this scenario? FP? TN? FN?
- TP:** The test predicted that the individual has COVID, and they do .
- FP:** The test predicted that the individual has COVID, but they don't .
- TN:** The test predicted that the individual doesn't have COVID, and they don't .
- FN:** The test predicted that the individual doesn't have COVID, but they do .

Accuracy of COVID tests

The results of 100 UCSD Health COVID tests are given below.

	Predicted Negative	Predicted Positive
Actually Negative	TN = 90 	FP = 1 
Actually Positive	FN = 8 	TP = 1 

UCSD Health test results

 **Question:** What is the accuracy of the test?

 **Answer:**
$$\text{accuracy} = \frac{TP + TN}{TP + FP + FN + TN} = \frac{1 + 90}{100} = 0.91$$

- Followup:** At first, the test seems good. But, suppose we build a classifier that predicts that **nobody has COVID**. What would its accuracy be?
- Answer to followup:** Also 0.91! There is severe **class imbalance** in the dataset, meaning that most of the data points are in the same class (no COVID). Accuracy doesn't tell the full story.

	Predicted Negative	Predicted Positive
Actually Negative	TN = 90 ✓	FP = 1 ✗
Actually Positive	FN = 8 ✗	TP = 1 ✓

UCSD Health test results

🤔 **Question:** What proportion of individuals who actually have COVID did the test identify?

🧐 **Answer:** $\frac{1}{1 + 8} = \frac{1}{9} \approx 0.11$

More generally, the **recall** of a binary classifier is the proportion of **actually positive instances** that are correctly classified. We'd like this number to be as close to 1 (100%) as possible.

$$\text{recall} = \frac{\text{\# you predicted positive}}{\text{\# actually positive}} = \frac{TP}{TP + FN}$$

To compute recall, look at the **bottom (positive) row** of the above confusion matrix.

Recall isn't everything, either!

$$\text{recall} = \frac{TP}{TP + FN}$$

🤔 **Question:** Can you design a "COVID test" with perfect recall?

🧐 **Answer:** Yes – **just predict that everyone has COVID!**

	Predicted Negative	Predicted Positive
Actually Negative	TN = 0 ✓	FP = 91 ✗
Actually Positive	FN = 0 ✗	TP = 9 ✓

everyone-has-COVID classifier

$$\text{recall} = \frac{TP}{TP + FN} = \frac{9}{9 + 0} = 1$$

Like accuracy, recall on its own is not a perfect metric. Even though the classifier we just created has perfect recall, it has 91 false positives!

Precision

	Predicted Negative	Predicted Positive
Actually Negative	TN = 0 ✓	FP = 91 ✗
Actually Positive	FN = 0 ✗	TP = 9 ✓

everyone-has-COVID classifier

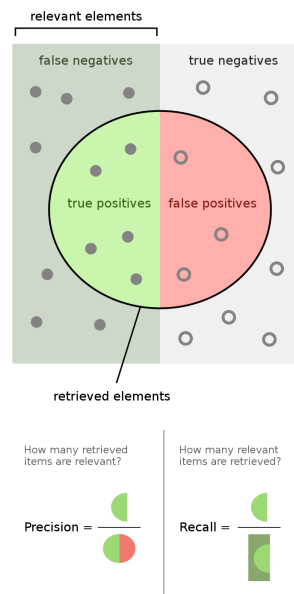
The **precision** of a binary classifier is the proportion of **predicted positive instances** that are correctly classified. We'd like this number to be as close to 1 (100%) as possible.

$$\text{precision} = \frac{\text{\# correct positive predictions}}{\text{\# you predicted positive}} = \frac{TP}{TP + FP}$$

To compute precision, look at the **right (positive) column** of the above confusion matrix.

- **Tip:** A good way to remember the difference between precision and recall is that in the denominator for **P**recision, both terms have **P** in them (TP and FP).
- Note that the "everyone-has-COVID" classifier has perfect recall, but a precision of $\frac{9}{9 + 91} = 0.09$, which is quite low.
- **Key idea:** There is a "tradeoff" between precision and recall. Ideally, you want both to be high. For a particular prediction task, one may be important than the other.

Precision and recall



(source)

Precision and recall

$$\text{precision} = \frac{TP}{TP + FP} \quad \text{recall} = \frac{TP}{TP + FN}$$

🤔 When might high **precision** be more important than high recall?

🤔 When might high **recall** be more important than high precision?

Question 🤔

Taken from the Spring 2022 Final Exam.

After fitting a `BillyClassifier`, we use it to make predictions on an unseen test set. Our results are summarized in the following confusion matrix.

	Predicted Negative	Predicted Positive
Actually Negative	???	30
Actually Positive	66	105

Part 1: What is the recall of our classifier? Give your answer as a fraction (it does not need to be simplified).

Part 2: The accuracy of our classifier is $\frac{69}{117}$. How many **true negatives** did our classifier have? Give your answer as an integer.

Part 3: True or False: In order for a binary classifier's precision and recall to be equal, the number of mistakes it makes must be an even number.

Part 4: Suppose we are building a classifier that listens to an audio source (say, from your phone's microphone) and predicts whether or not it is Soulja Boy's 2008 classic "Kiss Me thru the Phone." Our classifier is pretty good at detecting when the input stream is "Kiss Me thru the Phone", but it often incorrectly predicts that similar sounding songs are also "Kiss Me thru the Phone."

Complete the sentence: Our classifier has...

- low precision and low recall.
- low precision and high recall.
- high precision and low recall.
- high precision and high recall.

Computing with sklearn

- We can compute precision and recall with `sklearn.metrics`:

```
In [35]: from sklearn import metrics
```

Loading [MathJax]/extensions/Safe.js

```
# who actually has covid?
y_actual = np.random.uniform(size=100) < .1
y_actual.mean()
```

Out[35]: np.float64(0.06)

```
In [36]: # the predictions of three classifiers

# everyone has covid!
y_everyone = np.ones(100)

# nobody has covid!
y_nobody = np.zeros(100)

# some people have covid, determined by a test
y_test = y_actual.copy()

# the code below generates realistic test results; not perfect, but correlated

# flip 10% of the Trues to Falses, and 20% of the Falses to Trues
ix_true = np.nonzero(y_test)[0]
ix_false = np.nonzero(~y_test)[0]

flip_ix = np.random.choice(ix_true, size=int(0.1*len(ix_true)))
y_test[flip_ix] = False
flip_ix = np.random.choice(ix_false, size=int(0.2*len(ix_false)))
y_test[flip_ix] = True
```

```
In [37]: models = {
    "Lab Test": y_test,
    "Everyone": y_everyone,
    "Nobody": y_nobody
}
```

Accuracy

```
In [38]: for model, y_pred in models.items():
    score = metrics.accuracy_score(y_actual, y_pred)
    print(f'{model}: {score}')
```

Lab Test: 0.84

Everyone: 0.06

Nobody: 0.94

Note: the model that predicts that no one has COVID has the highest accuracy.

Precision

```
In [39]: for model, y_pred in models.items():
    score = metrics.precision_score(y_actual, y_pred)
    print(f'{model}: {score}')
```


Lab Test: 0.27272727272727

Everyone: 0.06

Nobody: 0.0

/Users/elldridge/workbench/dsc80/.venv/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning:

Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero\_division` parameter to control this behavior.

Recall

```
In [40]: for model, y_pred in models.items():
          score = metrics.recall_score(y_actual, y_pred)
          print(f'{model}: {score}')
```

Lab Test: 1.0

Everyone: 1.0

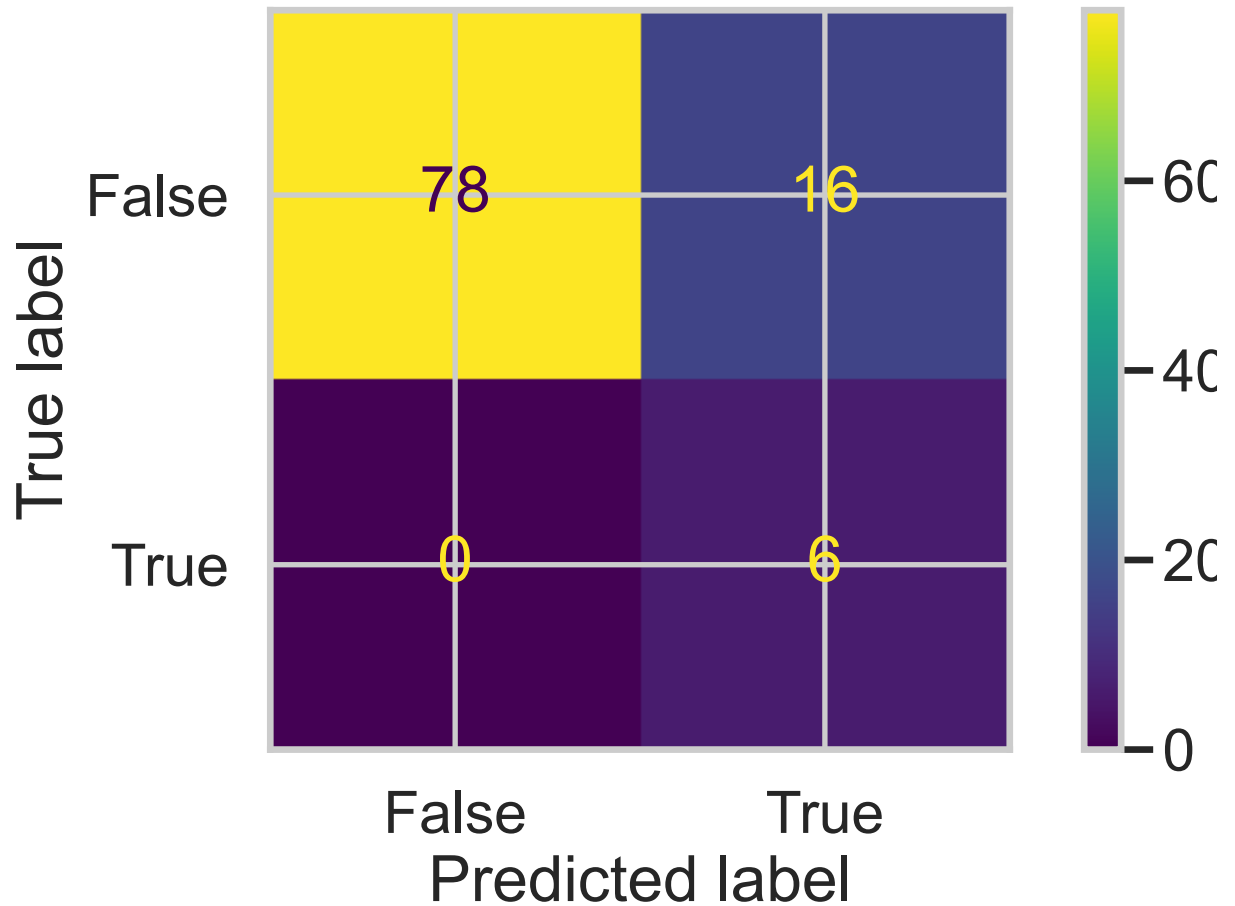
Nobody: 0.0

Note: the model that everyone has COVID has the highest recall.

Confusion Matrix

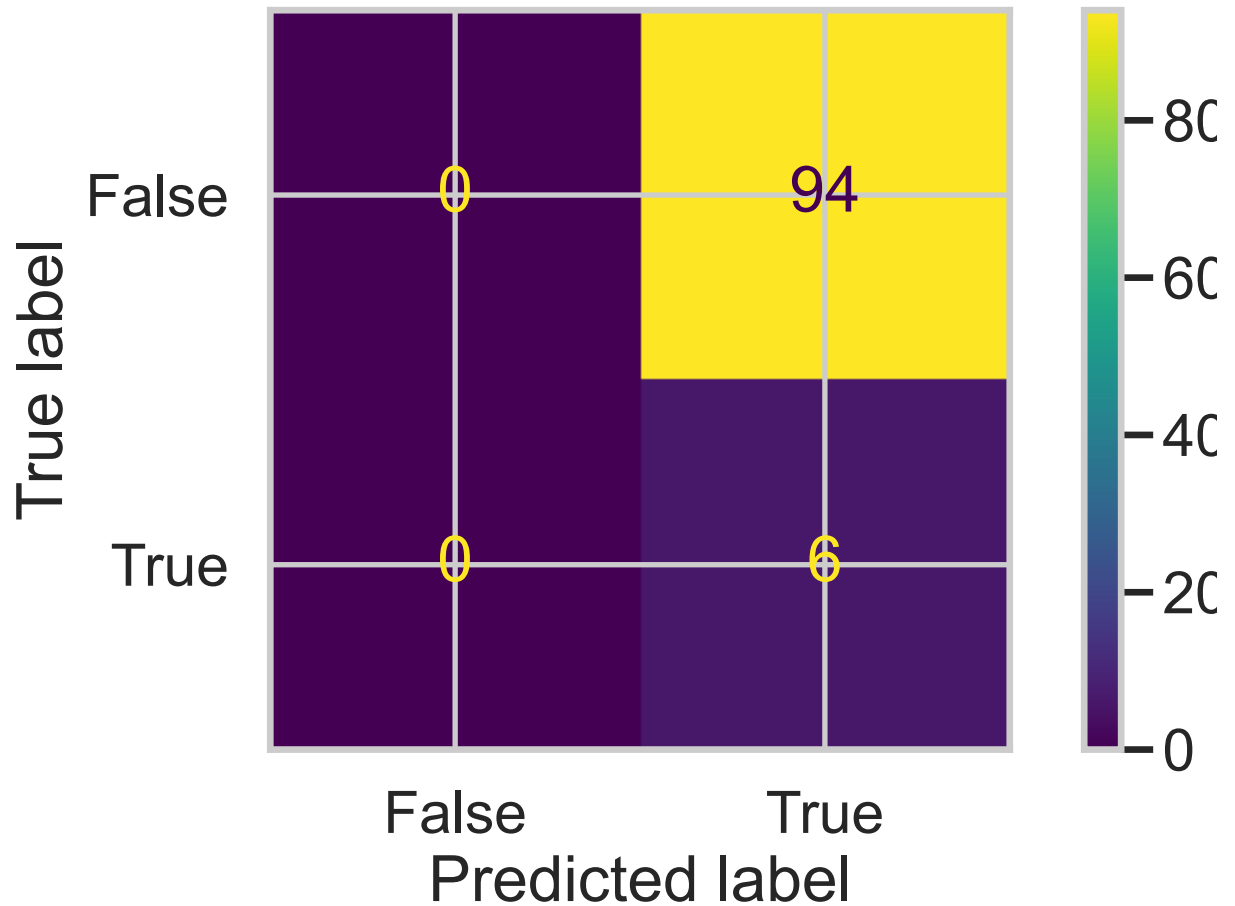
```
In [41]: metrics.ConfusionMatrixDisplay.from_predictions(y_actual, y_test)
```

```
Out[41]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1185fa1e0>
```



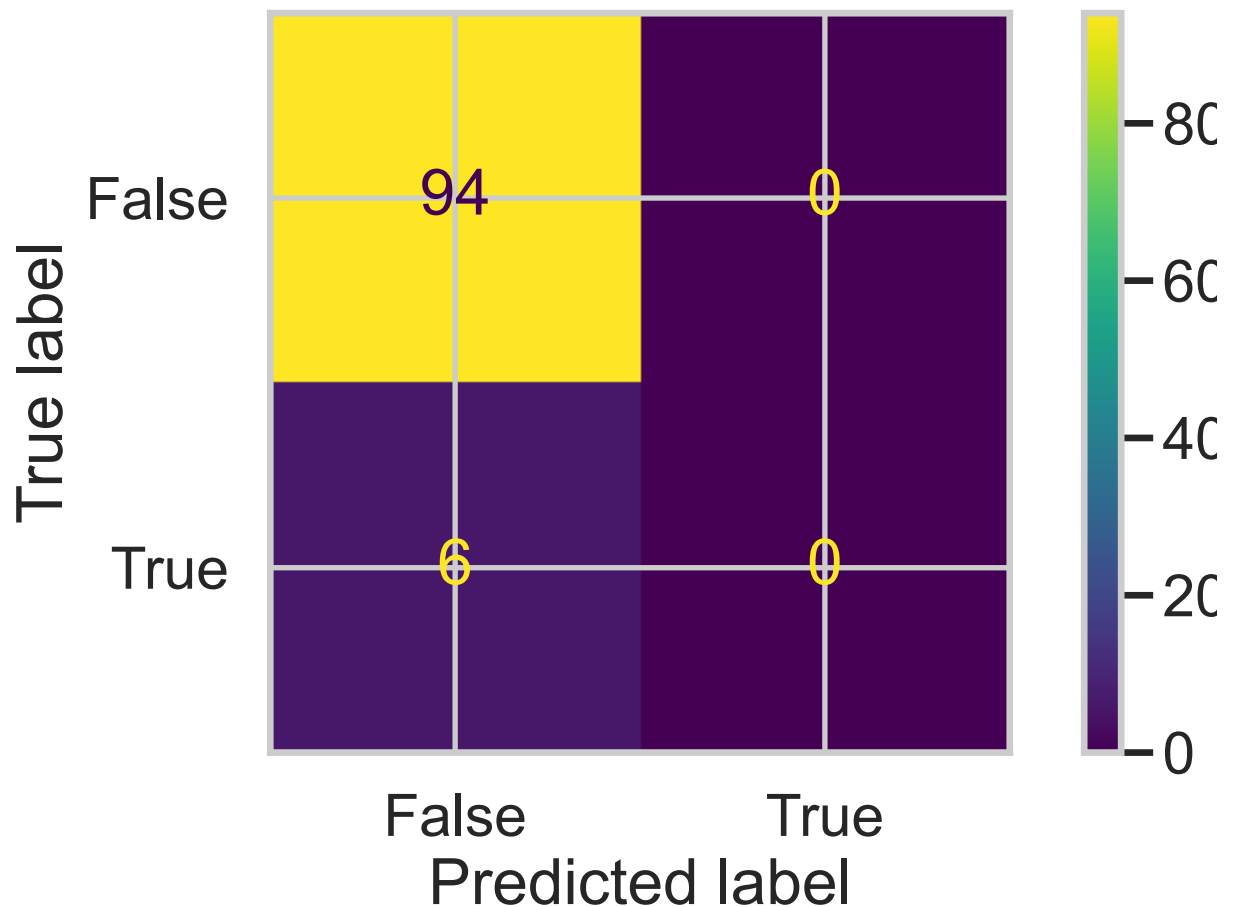
```
In [42]: metrics.ConfusionMatrixDisplay.from_predictions(y_actual, y_everyone)
```

```
Out[42]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x118183f20>
```



```
In [43]: metrics.ConfusionMatrixDisplay.from_predictions(y_actual, y_nobody)
```

```
Out[43]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x116a9c530>
```



Combining precision and recall

If we care equally about a model's precision PR and recall RE , we can combine the two using a single metric called the **F1-score**:

$$\text{F1-score} = \text{harmonic mean}(PR, RE) = 2 \frac{PR \cdot RE}{PR + RE}$$

```
In [44]: pr = metrics.precision_score(y_actual, y_test)
re = metrics.recall_score(y_actual, y_test)

2 * pr * re / (pr + re)
```

```
Out [44]: np.float64(0.42857142857142855)
```

```
In [45]: metrics.f1_score(y_actual, y_test)
```

```
Out [45]: np.float64(0.42857142857142855)
```

Both F1-score and accuracy are overall measures of a binary classifier's performance. But remember, accuracy is misleading in the presence of class imbalance, and doesn't take into account the kinds of errors the classifier makes.

```
In [46]: for model, y_pred in models.items():
score = metrics.f1_score(y_actual, y_pred)
```

```
print(f'{model}: {score}')
```

Lab Test: 0.42857142857142855

Everyone: 0.11320754716981132

Nobody: 0.0

Other evaluation metrics for binary classifiers

We just scratched the surface! This [excellent table from Wikipedia](#) summarizes the many other metrics that exist.

Sources: [1][2][3][4][5][6][7][8] view · talk · edit

		Predicted condition			
		Positive (PP)	Negative (PN)	Informedness, bookmaker informedness (BM) $= \text{TPR} + \text{TNR} - 1$	Prevalence threshold (PT) $= \frac{\sqrt{\text{TPR} \times \text{FPR}} - \text{FPR}}{\text{TPR} - \text{FPR}}$
Actual condition	Positive (P)	True positive (TP), hit	False negative (FN), type II error, miss, underestimation	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power $= \frac{\text{TP}}{P} = 1 - \text{FNR}$	False negative rate (FNR), miss rate $= \frac{\text{FN}}{P} = 1 - \text{TPR}$
	Negative (N)	False positive (FP), type I error, false alarm, overestimation	True negative (TN), correct rejection	False positive rate (FPR), probability of false alarm, fall-out $= \frac{\text{FP}}{N} = 1 - \text{TNR}$	True negative rate (TNR), specificity (SPC), selectivity $= \frac{\text{TN}}{N} = 1 - \text{FPR}$
	Prevalence $= \frac{P}{P+N}$	Positive predictive value (PPV), precision $= \frac{\text{TP}}{\text{TP} + \text{FP}} = 1 - \text{FDR}$	False omission rate (FOR) $= \frac{\text{FN}}{\text{FN} + \text{TN}} = 1 - \text{NPV}$	Positive likelihood ratio (LR+) $= \frac{\text{TPR}}{\text{FPR}}$	Negative likelihood ratio (LR-) $= \frac{\text{FNR}}{\text{TNR}}$
	Accuracy (ACC) $= \frac{\text{TP} + \text{TN}}{P + N}$	False discovery rate (FDR) $= \frac{\text{FP}}{\text{TP} + \text{FP}} = 1 - \text{PPV}$	Negative predictive value (NPV) $= \frac{\text{TN}}{\text{FN} + \text{TN}}$ $= 1 - \text{FOR}$	Markedness (MK), deltaP (Δp) $= \text{PPV} + \text{NPV} - 1$	Diagnostic odds ratio (DOR) $= \frac{\text{LR+}}{\text{LR-}}$
	Balanced accuracy (BA) $= \frac{\text{TPR} + \text{TNR}}{2}$	F₁ score $= \frac{2\text{PPV} \times \text{TPR}}{\text{PPV} + \text{TPR}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$	Fowlkes–Mallows index (FM) $= \sqrt{\text{PPV} \times \text{TPR}}$	Matthews correlation coefficient (MCC) $= \frac{\sqrt{\text{TPR} \times \text{TNR} \times \text{PPV} \times \text{NPV}}}{\sqrt{\text{FNR} \times \text{FPR} \times \text{FOR} \times \text{FDR}}}$	Threat score (TS), critical success index (CSI), Jaccard index $= \frac{\text{TP}}{\text{TP} + \text{FN} + \text{FP}}$

If you're interested in exploring further, a good next metric to look at is **true negative rate (i.e. specificity)**, which is the analogue of recall for true negatives.

Model fairness

Fairness: why do we care?

- Sometimes, a model performs better for certain groups than others; in such cases we say the model is **unfair**.
- Since ML models are now used in processes that significantly affect human lives, it is important that they are fair!
 - Job applications and college admissions.
 - Criminal sentencing and parole grants.
 - Predictive policing.
 - Credit and loans.

Model fairness

- We'd like to build a model that is *fair*, meaning that it performs the same for individuals within a group and individuals outside of the group.
- What do we mean by "perform"? What do we mean by "the same"?



par·i·ty¹

/ˈperədē/

noun

1. the state or condition of being equal, especially regarding status or pay.
"parity of incomes between rural workers and those in industrial occupations"

Similar:

equality

equivalence

uniformity

sameness

consistency



Parity measures for classifiers

Suppose C is a classifier we've already trained, and A is some binary attribute that denotes whether an individual is a member of a *sensitive* group – that is, a group we want to avoid discrimination for (e.g. $A = \{\text{age is less than 25}\}$).

- C achieves **accuracy parity** if C has the same accuracy for individuals in A and individuals not in A .
 - **Example:** C is a binary classifier that determines whether someone receives a loan.
 - If the classifier predicts correctly, then either C approves the loan and it is paid off, or C denies the loan and it would have defaulted.
 - If C achieves accuracy parity, then the proportion of correctly classified loans should be the same for those under 25 and those over 25.
- C achieves **precision (or recall) parity** if C has the same precision (or recall) for individuals in A and individuals not in A .
 - Recall parity is often called "true positive rate parity."
- C achieves **demographic parity** if the proportion of predictions that are positive is equal for individuals in A and individuals not in A .
- With the exception of demographic parity, the parity measures above all involve checking whether some evaluation metric from is equal across two groups.

- Which parity metric should you care about? It depends on your specific dataset and what types of errors are important!
- Many of these parity measures are **impossible** to satisfy simultaneously!
- The classifier parity metrics mentioned on the previous slide are only a few of the many possible parity metrics. See these [DSC 167 notes](#) for more details, including more formal explanations.
- These don't apply for regression models; for those, we may care about **RMSE parity** or **R^2 parity**. There is also a notion of demographic parity for regression models, but it is outside of the scope of DSC 80.

Example: Loan approval

As you know from Project 2, LendingClub was a "peer-to-peer lending company"; they [used to publish](#) a dataset describing the loans that they approved.

- `'tag'` : whether loan was repaid in full (1.0) or defaulted (0.0).
- `'loan_amnt'` : amount of the loan in dollars.
- `'emp_length'` : number of years employed.
- `'home_ownership'` : whether borrower owns (1.0) or rents (0.0).
- `'inq_last_6mths'` : number of credit inquiries in last six months.
- `'revol_bal'` : revolving balance on borrows accounts.
- `'age'` : age in years of the borrower (protected attribute).

```
In [47]: loans = pd.read_csv(Path('data') / 'loan_vars1.csv', index_col=0)
         loans.head()
```

```
Out [47]:
```

	loan_amnt	emp_length	home_ownership	inq_last_6mths	revol_bal	ag
268309	6400.000	0.000	1.000	1.000	899.000	22.00
301093	10700.000	10.000	1.000	0.000	29411.000	19.00
1379211	15000.000	10.000	1.000	2.000	9911.000	48.00
486795	15000.000	10.000	1.000	2.000	15883.000	35.00
1481134	22775.000	3.000	1.000	0.000	17008.000	39.00

The total amount of money loaned was over 5 billion dollars!

```
Loading [MathJax]/extensions/Safe.js
loans['loan_amnt'].sum()
```

```
Out [48]: np.float64(5706507225.0)
```

```
In [49]: loans.shape[0]
```

```
Out [49]: 386772
```

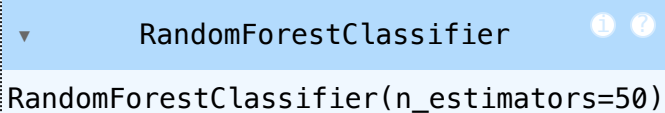
Predicting 'tag'

Let's build a classifier that predicts whether or not a loan was paid in full. If we were a bank, we could use our trained classifier to determine whether to approve someone for a loan!

```
In [50]: from sklearn.model_selection import train_test_split  
from sklearn.ensemble import RandomForestClassifier
```

```
In [51]: X = loans.drop('tag', axis=1)  
y = loans.tag  
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

```
In [52]: clf = RandomForestClassifier(n_estimators=50)  
clf.fit(X_train, y_train)
```

```
Out [52]: RandomForestClassifier(n_estimators=50)
```

Remember, a prediction of 1 means that we predict that the loan will be paid in full.

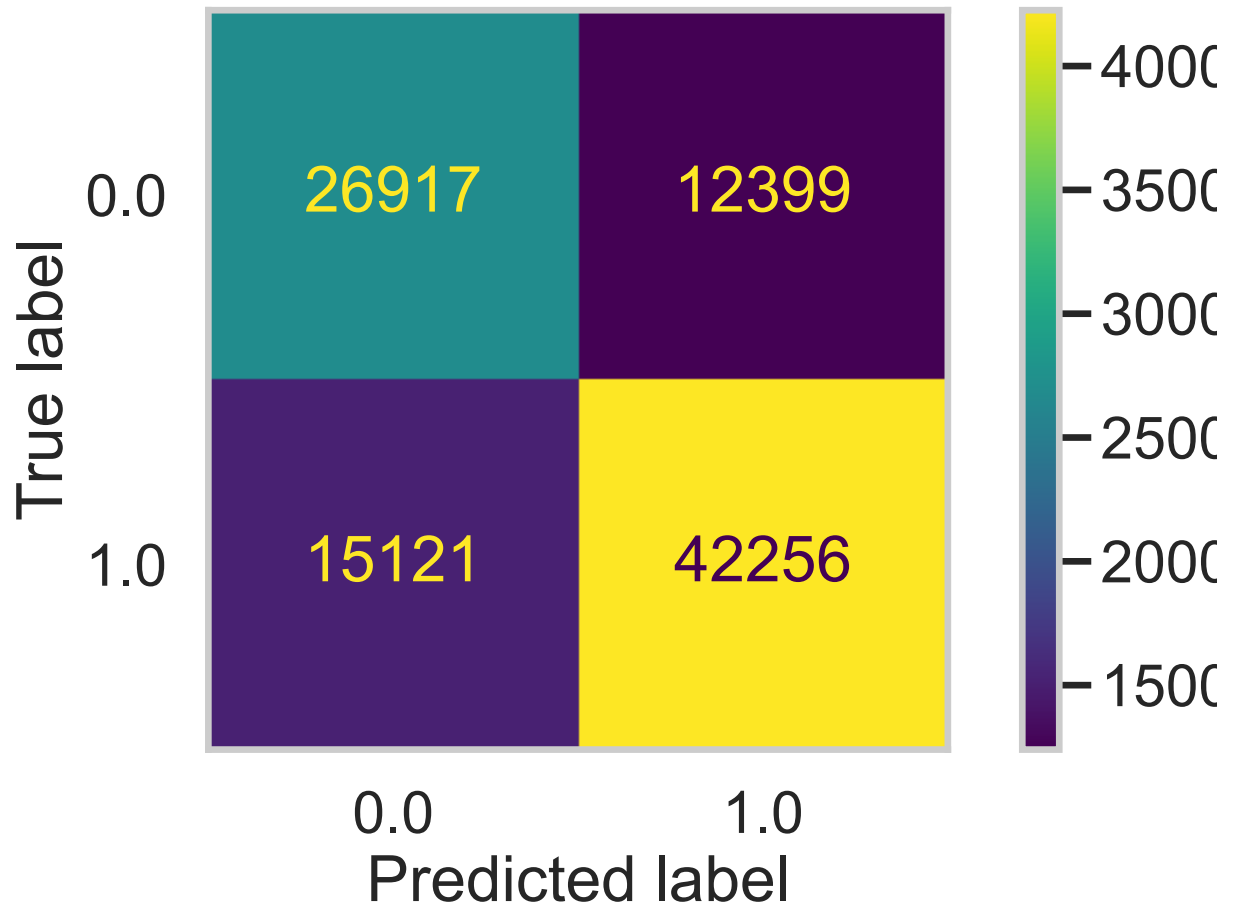
```
In [53]: y_pred = clf.predict(X_test)  
y_pred
```

```
Out [53]: array([1., 0., 1., ..., 1., 0., 1.])
```

```
In [54]: clf.score(X_test, y_test)
```

```
Out [54]: 0.7153878770955499
```

```
In [55]: metrics.ConfusionMatrixDisplay.from_estimator(clf, X_test, y_test);  
plt.grid(False)
```

Precision

$$\text{precision} = \frac{TP}{TP + FP}$$

Precision describes the **proportion of loans that were approved that would have been paid back**.

```
In [56]: metrics.precision_score(y_test, y_pred)
```

```
Out [56]: np.float64(0.77314060927637)
```

If we subtract the precision from 1, we get the proportion of loans that were approved that **would not** have been paid back. This is known as the **false discovery rate**.

$$\frac{FP}{TP + FP} = 1 - \text{precision}$$

```
In [57]: 1 - metrics.precision_score(y_test, y_pred)
```

```
Out [57]: np.float64(0.22685939072363004)
```

Recall

$$\text{recall} = \frac{TP}{TP + FN}$$

Loading [MathJax]/extensions/Safe.js

Recall describes the **proportion of loans that would have been paid back that were actually approved**.

```
In [58]: metrics.recall_score(y_test, y_pred)
```

```
Out [58]: np.float64(0.7364623455391534)
```

If we subtract the recall from 1, we get the proportion of loans that would have been paid back that **were denied**. This is known as the **false negative rate**.

$$\frac{FN}{TP + FN} = 1 - \text{recall}$$

```
In [59]: 1 - metrics.recall_score(y_test, y_pred)
```

```
Out [59]: np.float64(0.26353765446084665)
```

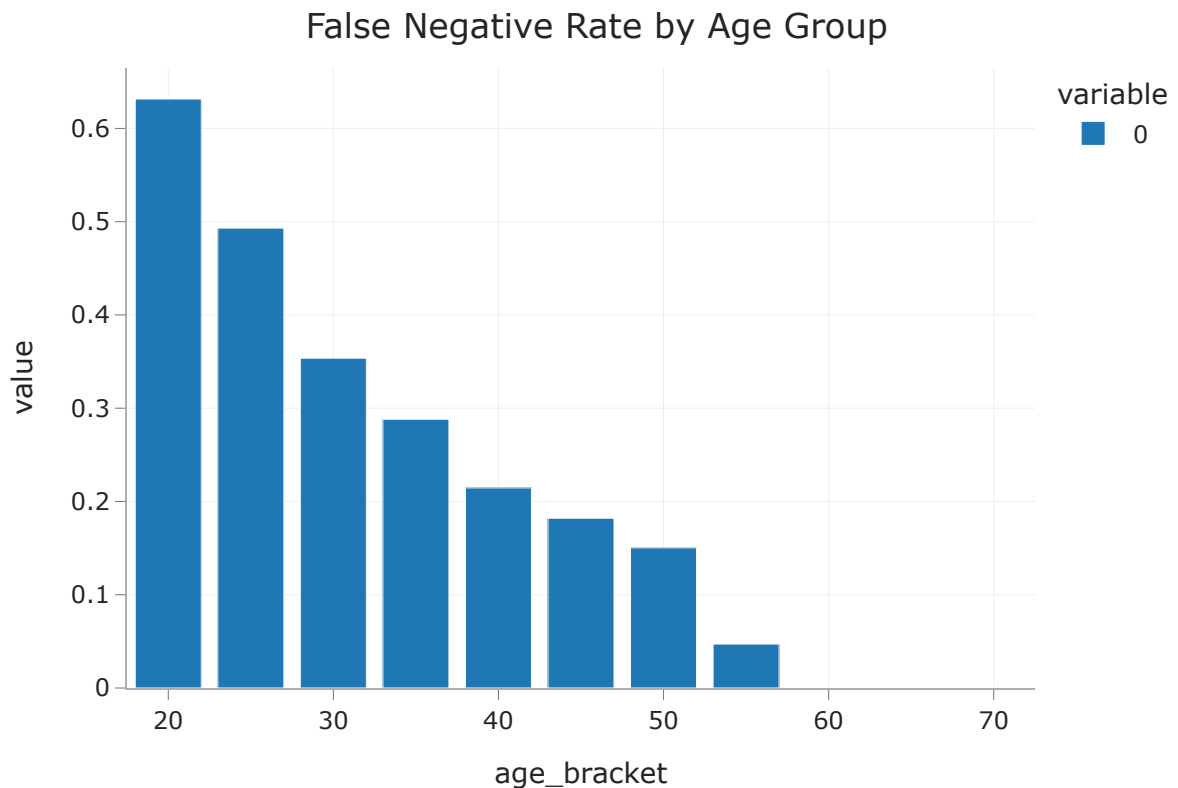
From both the perspective of the bank and the lendee, a high false negative rate is bad!

- The bank left money on the table – the lendee would have paid back the loan, but they weren't approved for a loan.
- The lendee deserved the loan, but weren't given one.

False negative rate by age

```
In [60]: results = X_test
results['age_bracket'] = results['age'].apply(lambda x: 5 * (x // 5 + 1))
results['prediction'] = y_pred
results['tag'] = y_test

(
    results
    .groupby('age_bracket')
    [['tag', 'prediction']]
    .apply(lambda x: 1 - metrics.recall_score(x['tag'], x['prediction']))
    .plot(kind='bar', title='False Negative Rate by Age Group')
)
```



Computing parity measures

- $\$C\$$: Our random forest classifier (1 if we approved the loan, 0 if we denied it).
- $\$A\$$: Whether or not they were under 25 (1 if under, 0 if above).

```
In [61]: results['is_young'] = (results['age'] < 25).replace({True: 'young', False: 'old'})
```

First, let's compute the proportion of loans that were approved in each group. If these two numbers are the same, $\$C\$$ achieves demographic parity.

```
In [62]: results.groupby('is_young')['prediction'].mean()
```

```
Out[62]: is_young
old      0.686
young    0.301
Name: prediction, dtype: float64
```

$\$C\$$ evidently does not achieve demographic parity – older people are approved for loans far more often! Note that this doesn't factor in whether they were *correctly* approved or *incorrectly* approved.

Now, let's compute the accuracy of $\$C\$$ in each group. If these two numbers are the same, $\$C\$$ achieves accuracy parity.

```
In [63]: compute_accuracy = lambda x: metrics.accuracy_score(x['tag'], x['prediction'])
```

Loading [MathJax]/extensions/Safe.js

```
In [64]: (
    results
    .groupby('is_young')
    [['tag', 'prediction']]
    .apply(compute_accuracy)
    .rename('accuracy')
)
```

```
Out[64]: is_young
old      0.731
young    0.680
Name: accuracy, dtype: float64
```

Hmm... These numbers look much more similar than before!

Is this difference in accuracy significant?

Let's run a **permutation test** to see if the difference in accuracy is significant.

- **Null Hypothesis:** The classifier's accuracy is the same for both young people and old people, and any differences are due to chance.
- **Alternative Hypothesis:** The classifier's accuracy is higher for old people.
- Test statistic: Difference in accuracy (young minus old).
- Significance level: 0.01.

```
In [65]: obs = (results
    .groupby('is_young')
    [['tag', 'prediction']]
    .apply(compute_accuracy)
    .diff()
    .iloc[-1])

obs
```

```
Out[65]: np.float64(-0.050863360439697525)
```

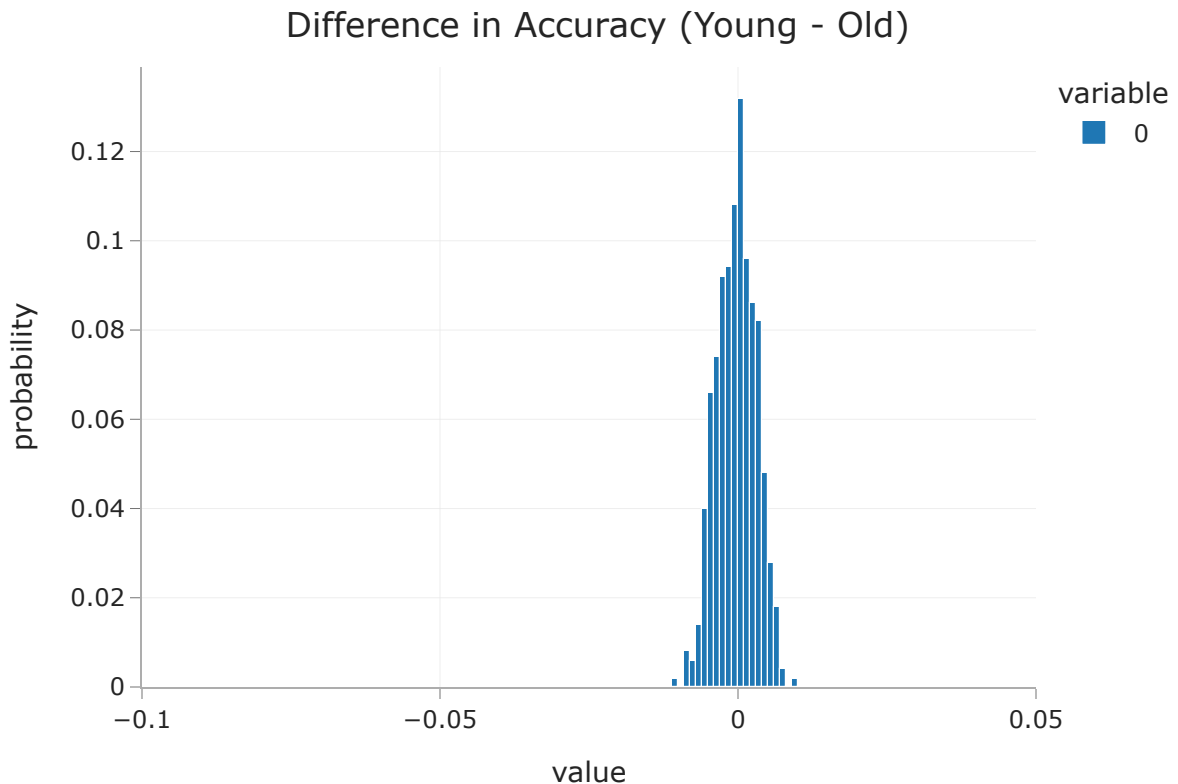
```
In [66]: diff_in_acc = []
for _ in range(500):
    s = (
        results[['is_young', 'prediction', 'tag']]
        .assign(is_young=np.random.permutation(results['is_young']))
        .groupby('is_young')
        [['tag', 'prediction']]
        .apply(compute_accuracy)
        .diff()
        .iloc[-1]
    )

    diff_in_acc.append(s)
```

```
In [67]: fig = pd.Series(diff_in_acc).plot(kind='hist', histnorm='probability', nbins
    title='Difference in Accuracy (Young - Old)')
```

Loading [MathJax]/extensions/Safe.js

```
fig.add_vline(x=obs, line_color='red')
fig.update_layout(xaxis_range=[-0.1, 0.05])
```



It seems like the difference in accuracy across the two groups **is significant**, despite being only ~5%. Thus, $\$C\$$ likely does not achieve accuracy parity.

Ethical questions of fairness

- **Question:** Is it "fair" to deny loans to younger people at a higher rate?
- This is not a *data science* question, it is a political and philosophical question.
 - *But* that doesn't mean that data scientists can ignore it!
- Federal law prevents age from being used as a determining factor in denying a loan.

Not only should we **not** use `'age'` to determine whether or not to approve a loan, but we also shouldn't use other features that are strongly correlated with `'age'`, like `'emp_length'`.

In [68]: `loans`

Out [68]:

	loan_amnt	emp_length	home_ownership	inq_last_6mths	revol_bal	ag
268309	6400.000	0.000	1.000	1.000	899.000	22.00
301093	10700.000	10.000	1.000	0.000	29411.000	19.00
1379211	15000.000	10.000	1.000	2.000	9911.000	48.00
...	...	...	...	...	...	...
1150493	5000.000	1.000	1.000	0.000	3842.000	52.00
686485	6000.000	10.000	0.000	0.000	6529.000	36.00
342901	15000.000	8.000	1.000	1.000	16060.000	39.00

386772 rows x 7 columns



Fin

- Thanks!
- Keep in touch!
- Good luck on finals, and have a good break.

In []: